

Shover-World: Environment and GUI Documentation

Milestone 1 Report

November 2025

Abstract

This document explains the logic behind the Shover-World project. The project is a grid-based game where boxes are pushed. It has two main parts: the environment, which handles the rules and math, and the GUI, which shows the game on the screen. This report describes how the movement, energy (stamina) costs, special box patterns, and the visual interface work.

I. Introduction

In this project `environment.py`, acts as the brain of the game. It keeps track of where everything is and calculates what happens when the player moves. In addition, `gui.py`, acts as the face of the game. It uses the Pygame library to draw pictures and let the player use the keyboard.

II. Environment Logic

a. Initialization and Configuration

The environment starts with specific settings that define the size of the map and the "energy" rules for the agent.

Table 1: Initialization Parameters (environment.py)

Parameter	Type	Description
n_rows, n_cols	int	Size of the map ($a \times b$)
max_timestep	int	Max turns allowed (default: 400)
initial_stamina	float	Starting energy ($S_0 = 1000.0$)
initial_force	float	Cost to start moving a stopped box
unit_force	float	Cost per box being pushed
perf_sq_initial_age	int	Turns before a square disappears
edge_lava	bool	If the edge of the map is lava

b. The Grid and State

The game world is a grid of numbers. We use a matrix (a list of rows and columns) to store the map. Each number represents an object.

Table 2: Grid Number Meanings

Value	Object
-100	Lava (Danger zone)
0	Empty Space
1	Barrier (Wall)
1 – 10	Box (Pushable Object)
100	Barrier

c. Movement and Physics

When the agent tries to move, the code checks what is in front:

- **Empty Space:** The box simply can be moved there.
- **Wall:** The move is blocked. Nothing happens.
- **Box:** The code checks for a "chain" of boxes. It looks ahead to see how many boxes are touching each other in a line.

If there is a chain of boxes, the code checks the space after the last box. If that space is empty, all the boxes move one step. If that space is a wall or another box, the whole chain is stuck and cannot move.

d. Lava Logic

There is a special rule for lava. If a box is pushed into a lava cell (-100), the box is destroyed. It is removed from the grid (becomes 0). When this happens, the

player gets some stamina back as a refund. This rewards the player for clearing the map.

III. Stamina Mechanics

The game uses a "Stamina" system. Every action costs energy. If stamina reaches zero, the game ends. The cost is calculated using simple physics logic:

- **Basic Step:** Just walking costs a small amount (1.0).
- **Pushing Weight:** Pushing boxes costs more. The more boxes you push at once, the higher the cost (Unit Force).
- **Starting Friction:** The code remembers if a box was moving in the last turn. If a box is stopped, it takes extra energy to start moving it (Initial Force). If it is already moving, this extra cost is skipped.

The mathematical formula used in the code is:

$$\Delta S_{push} = F_{initial} \cdot \mathbb{I}_{stationary} + F_{unit} \cdot k$$

Where:

- $F_{initial}$ = The cost to start moving a stopped box.
- $\mathbb{I}_{stationary}$ = 1 if the box was stopped, 0 if it was moving.
- F_{unit} = The cost per box.
- k = How many boxes are in the chain.

IV. Special Actions (Perfect Squares)

The code constantly scans the grid to find "Perfect Squares." A Perfect Square is a block of boxes (like 2×2 or 3×3) with no other boxes touching the outside edges.

If the code finds these squares, the player can use special keys:

- **Barrier Maker:** Turns a square of boxes into walls. This gives the player a lot of stamina.
- **Hellify:** Turns the inside of a big square into lava and removes the outside border.

The code also tracks the "age" of these squares. If the player ignores a perfect square for too long, it automatically disappears.

V. Graphical User Interface (GUI)

The `gui.py` file makes the game visible. It runs a loop that listens to inputs, connects to the game logic, and draws the screen.

When the game starts, the user sees the **Main Menu** first. In this menu, the player can choose to play on a **Random Map**, **Map 1**, or **Map 2**. After selecting an option, the player enters the game environment.

Figure 2 shows the design of the menu screen.



Figure 1: The Main Menu screen where the player selects a map.

a. Drawing the Game

The GUI looks at the grid numbers. If it sees a 100, it draws a wall image. If it sees a box number, it draws a box image. It also writes the current Stamina and Time at the top of the screen so the player knows their status.

b. Inputs

The input handling maps keyboard buttons to game actions.



Figure 2: The Pygame Screen

Table 3: Controls

Key	Action
Arrow Keys / WASD	Move or Push
B	Barrier Maker
H	Hellify
R	Reset the Game
Q / ESC	Quit
Mouse Click	Select a target box

VI. Testing

To ensure the game logic is robust and bug-free, we implemented a complete suite of unit tests using the `pytest` framework. The testing process covers four main areas of the project:

- **Map Loading:** We verify that the game correctly interprets map data

(both integer and symbolic formats) and raises errors for invalid inputs, such as malformed rows or unknown symbols.

- **Movement Physics:** These tests check that the agent pushes boxes correctly, handles chain reactions, and stops when blocked by walls.
- **Pattern Detection:** We test the algorithms responsible for identifying "Perfect Squares" (e.g., 2×2 or 3×3 blocks of boxes), ensuring that adjacent clutter correctly invalidates a square.
- **Stamina & Abilities:** These tests verify the economy of the game. We check that stamina costs decrease when pushing moving boxes (momentum), and that special abilities like "Barrier Maker" and "Hellify" apply the correct changes to the grid.

We executed the full test suite, and all 15 tests passed successfully. Figure 3 shows the successful execution of these tests.

```
tests\test_maps.py ..... [ 33%]
tests\test_push.py .. [ 66%]
tests\test_squares.py ... [ 83%]
tests\test_stamina.py ..... [100%]
===== 15 passed in 0.31s =====
```

Figure 3: Terminal output showing that all 15 unit tests passed.

VII. Conclusion

The Shover-World project successfully implements a grid game with complex rules. The `environment.py` handles the math for movement, stamina costs, and special box patterns. The `gui.py` provides a simple way to play and test these mechanics using Pygame. The separation of logic and graphics makes the code clean and easy to test.